kruskal

人员

于家瑞、陶汇笙、于霄龙、王恩泽、崔宸赫、郭栩睿 到课

上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3190

比赛概况 题目列表 选择题列表 提交记录 实时榜单 选择题排行榜 编辑比赛

王向东老师周六十点半C++dijkstra

刷新

#	用户名	姓名	编程分	时间	A	B	C
1	wangenze	王恩泽	200	1133	0	100	100
2	taohuisheng	陶汇笙	200	1647	100	100	
3	guoxurui	郭栩睿	200	2291	100	100	
4	yujiarui	于家瑞	200	2327	100	100	
5	yuxiaolong	于霄龙	200	2328	100	100	
6	hongchendong	洪晨栋	200	2351	100	100	
7	cuichenhe	崔宸赫	200	2372	100	100	
8	hongchenqi	洪晨棋	160	742	60	100	

本周作业

https://cppoj.kids123code.com/contest/3296 (课上讲了 A ~ E 这些题, 课后作业是 E 题)

课堂表现

今天讲了 kruskal 和 快速幂 这两个知识

在今天上课中, 反映出许多同学之前的并查集基础很不扎实, 同学们课下需要复习一下之前的并查集代码, 把并查集代码背熟

课堂内容

【模板】最小生成树

kruskal 模板题

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e5 + 5;
int f[maxn];
struct node {
  int u, v, value;
```

```
  bool operator < (const node& p) const { return value < p.value; }
};

int fFind(int x) {
  if (f[x] != x) f[x] = fFind(f[x]);
  return f[x];
}

void kruskal(vector<node>& vec, int n) {
  sort(vec.begin(), vec.end());
  for (int i = 1; i <= n; ++i) f[i] = i;

  int sum = 0, cnt = 0;
  for (node it : vec) {
    int u = it.u, v = it.v, value = it.value;
    int pu = fFind(u), pv = fFind(v);
    if (pu == pv) continue;
    f[pu] = pv; sum += value; ++cnt;
  }
  if (cnt != n-1) cout << "orz" << endl;
  else cout << sum << endl;
}

int main()
{
  int n, m; cin >> n >> m;
  vector<node> vec;
  while (m -- ) {
    int u, v, value; cin >> u >> v >> value;
    vec.push_back({u, v, value});
  }
  kruskal(vec, n);
  return 0;
}
```

[SCOI2005] 繁忙的都市

之前做过, 第一次做的方法是用 二分 + 并查集 做的, 第二个方法是直接构建最小生成树即可

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e5 + 5;
int f[maxn];
struct node {
  int u, v, value;
  bool operator < (const node& p) const { return value < p.value; }
};

int fFind(int x) {
```

```
  if (f[x] != x) f[x] = fFind(f[x]);
  return f[x];
}

void solve(vector<node>& vec, int n) {
  sort(vec.begin(), vec.end());
  for (int i = 1; i <= n; ++i) f[i] = i;

  int cnt = 0;
  for (node it : vec) {
    int u = it.u, v = it.v, value = it.value;
    int pu = fFind(u), pv = fFind(v);
    if (pu == pv) continue;
    f[pu] = pv; ++cnt;
    if (cnt == n-1) { cout << n-1 << " " << value << endl; break; }
  }
}

int main()
{
  int n, m; cin >> n >> m;
  vector<node> vec;
  while (m -- ) {
    int u, v, value; cin >> u >> v >> value;
    vec.push_back({u, v, value});
  }
  solve(vec, n);
  return 0;
}
```

Choose Two and Eat One

快速幂模板

```
int qmod(int a, int k, int mod) {
  int res = 1;
  while (k) {
    if (k&1) res = res*a % mod;
    a = a*a % mod;
    k >>= 1;
  }
  return res;
}
```

可以先 O(n^2) 求出任意两点间的得分, 然后构建最大生成树即可

```
#include <bits/stdc++.h>
#define int long long
```

```cpp
using namespace std;

const int maxn = 500 + 5;
struct node {
  int u, v, value;
  bool operator < (const node& p) const { return value < p.value; }
};
int w[maxn], f[maxn];

int fFind(int x) {
  if (f[x] != x) f[x] = fFind(f[x]);
  return f[x];
}

int qmod(int a, int k, int mod) {
  int res = 1;
  while (k) {
    if (k&1) res = res*a % mod;
    a = a*a % mod;
    k >>= 1;
  }
  return res;
}

signed main()
{
  int n, mod; cin >> n >> mod;
  for (int i = 1; i <= n; ++i) cin >> w[i];

  vector<node> edges;
  for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= n; ++j) {
      if (i != j) {
        int v = (qmod(w[i],w[j],mod) + qmod(w[j],w[i],mod)) % mod;
        edges.push_back({i, j, v});
      }
    }
  }

  sort(edges.begin(), edges.end()); reverse(edges.begin(), edges.end());
  for (int i = 1; i <= n; ++i) f[i] = i;

  int res = 0;
  for (node it : edges) {
    int u = it.u, v = it.v, value = it.value;
    int fu = fFind(u), fv = fFind(v);
    if (fu != fv) f[fu] = fv, res += value;
  }
  cout << res << endl;
  return 0;
}
```